TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant (LangGraph + HITL)

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is a hybrid AI architecture that integrates:

- **Retrieval Systems** → Fetch relevant external knowledge
- **Generative Models (LLMs)** → Generate human-like responses

Unlike traditional LLM systems that rely only on **pre-trained knowledge**, RAG enhances the response generation process by **injecting real-time contextual information** from external data sources.

🔹 **Key Idea:**

Instead of:

"Model answers from memory"

RAG does:

"Retrieve → Augment → Generate"

🔹 **Advantages of RAG:**

- Reduces hallucination
- Improves factual accuracy
- Supports domain-specific knowledge
- Enables dynamic updates without retraining

1.2 Why RAG is Needed

Traditional chatbot systems suffer from:

🔴 **Limitations:**

- Static and outdated knowledge
- Lack of contextual understanding
- Poor handling of domain-specific queries
- High dependency on human agents

✅ **How RAG Solves This:**

Problem	RAG Solution
Static knowledge	Dynamic retrieval
Generic answers	Context-aware responses
Hallucination	Grounded responses
Poor scalability	Automated system

1.3 Use Case Overview

The system is designed as an **AI-powered Customer Support Assistant** capable of:

- Understanding natural language queries
- Retrieving relevant information from PDF knowledge base
- Generating precise responses
- Escalating complex queries to human agents

---

2. System Architecture Explanation

2.1 Architecture Overview

The system follows a **layered modular architecture**, ensuring:

- Scalability
- Maintainability
- Clear separation of concerns

🔹 **Layers Explained:**

1. **Document Processing Layer**
   - Extracts and preprocesses data
2. **Embedding & Storage Layer**
   - Converts text into vector format
   - Stores embeddings
3. **Retrieval Layer**
   - Performs similarity search
4. **LLM Processing Layer**
   - Generates responses
5. **Workflow Layer (LangGraph)**
   - Controls execution logic
6. **Decision Layer**
   - Applies intelligence to route responses

7. **HITL Layer**
    - Ensures human intervention when required

2.2 Component Interaction (Expanded Flow)

```
PDF → Loader → Chunking → Embeddings → ChromaDB
                          ↑
User Query → Retriever → Context → LLM → Decision → Output
                          ↓
                         HITL
```

🔹 **Step-by-Step Execution:**

1. PDF is ingested and processed
2. Text is chunked into manageable units
3. Chunks are converted into embeddings
4. Stored in vector database
5. User query is received
6. Query is embedded and matched
7. Relevant chunks retrieved
8. LLM generates response
9. Decision layer evaluates quality
10. Response is returned or escalated

2.3 Key Architectural Feature (LangGraph)

LangGraph introduces **graph-based workflow orchestration**:

🔹 **Why LangGraph?**

- Enables **stateful workflows**
- Supports **conditional routing**
- Provides **modular execution nodes**

---

3. Design Decisions

3.1 Chunk Size Selection

- Chunk Size: 500 tokens
- Overlap: 50 tokens

🔹 **Why this matters:**

- LLMs have token limits
- Too large → slow + noisy

- Too small → loss of context

✅ **Balanced Choice:**

Ensures:

- Context preservation
- Efficient retrieval

3.2 Embedding Strategy

Used **semantic embeddings** instead of keyword matching.

🔹 **Benefit:**

- Captures meaning, not exact words
- Improves relevance of results

3.3 Retrieval Strategy

Top-K Retrieval (k = 3–5)

🔹 **Reason:**

- Too many chunks → noise
- Too few → incomplete context

3.4 Prompt Design Logic

Prompt engineering ensures:

- No hallucination
- Context grounding
- Clear instructions

🔹 **Example Strategy:**

- Provide context
- Restrict answer scope
- Add fallback instruction

---

4. Workflow Explanation (LangGraph)

4.1 Node Responsibilities (Detailed)

- **Retrieval Node**
  - Performs semantic search
  - Returns top-k chunks
- **Generation Node**
  - Combines context + query
  - Calls LLM
- **Decision Node**
  - Evaluates:
    - Confidence
    - Context availability
- **HITL Node**
  - Handles escalation
  - Integrates human response

4.2 State Management

State object ensures **data continuity**:

```
{
  "query": "",
  "retrieved_docs": [],
  "response": "",
  "confidence": 0.0,
  "escalation": false
}
```

4.3 State Transitions

- Each node updates state
- Decision node determines final flow

---

5. Conditional Logic

5.1 Intent Detection

System evaluates:

- Query complexity

- Context relevance
- LLM response quality

5.2 Routing Logic

Case 1: High Confidence

→ Direct Answer

Case 2: Low Confidence

→ HITL

5.3 Decision Criteria (Advanced)

- Confidence < 0.7
- No retrieved chunks
- Ambiguous query
- Multi-step reasoning

---

6. HITL Implementation

6.1 Role of HITL

HITL ensures:

- System reliability
- Reduced risk of wrong answers
- Better user trust

6.2 Escalation Conditions

- Low confidence
- Missing data
- Sensitive queries

6.3 HITL Workflow

Query → Flagged → Sent to Human → Human Response → Final Output

6.4 Benefits

- Improves accuracy
- Ensures quality control
- Handles edge cases

6.5 Limitations

- Slower response
- Requires human availability

---

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs Speed

- More data → better accuracy, slower
- Less data → faster, less accurate

7.2 Chunk Size Trade-off

- Large chunks → rich context
- Small chunks → better precision

7.3 Cost vs Performance

- LLM calls are expensive
- Need optimization

7.4 Latency

- Multi-step pipeline adds delay
- Needs caching & optimization

---

8. Testing Strategy

8.1 Testing Types

- Functional Testing
- Integration Testing
- Edge Case Testing

8.2 Sample Queries

- Refund policy
- Cancellation process
- Terms and conditions

8.3 Expected Behavior

- Correct answer for known queries

- “I don’t know” for unknown
- HITL for complex queries

---

9. Future Enhancements

9.1 Multi-Document Support

- Multiple PDFs
- Domain expansion

9.2 Feedback Loop

- Store user feedback
- Improve model performance

9.3 Memory Integration

- Maintain conversation history
- Context-aware dialogues

9.4 Deployment

- Web interface (Streamlit)
- Cloud deployment
- API integration

---

10. Conclusion

This project demonstrates a **real-world AI system design** by integrating:

- Retrieval-Augmented Generation (RAG)
- Graph-based workflow orchestration (LangGraph)
- Intelligent decision-making
- Human-in-the-loop escalation

✅ **Key Outcomes:**

- Improved accuracy
- Context-aware responses
- Scalable architecture
- Reliable AI-human collaboration